

SOFTWARE ENGINEERING PRACTICE-I

MIDTERM REPORT

OpenNutri: LLM-Assisted Extraction of Food Data from Scientific Literature

Students:

- 221229078 - Arciel Aliognis Baez Zamora
- 221229075 - Duc Huan Ngo
- 221229031 - Ayşegül Doğan

Advisor: Dr. Öğr. Üyesi Fatma Zehra Solak

Exam Date: [To be filled by the advisor]

SUMMARY OF WORK COMPLETED DURING THE TERM

This midterm report summarizes the current state of the first three work items defined for the first semester in the project proposal:

- automated data acquisition pipeline,
- database and orchestration architecture,
- expert annotation engine.

By the midterm stage, OpenNutri has a working core system. It discovers candidate papers from scientific sources, filters them through multiple stages, imports suitable PDFs, provides a structured annotation interface to the expert user, and feeds user decisions back into later crawling decisions.

The project flow is straightforward: the system finds papers, the expert user enters structured data on top of the document, and those labels improve later search and ranking. The main parts of that loop are already working together.

Table 1 summarizes how the current system state maps to the first-semester proposal items.

Proposal item	Midterm status	Concrete output
1. Automated Data Acquisition Pipeline	Substantially progressed	Multi-source search, language-scoped workflows, PDF acquisition, validation, upload to Supabase
2. Database and Orchestration Architecture	Substantially progressed	Shared schema, RLS policies, storage path, ETL and search-evidence tables
3. Expert Annotation Engine	Substantially progressed	PDF viewing, nutrient highlighting, dynamic food/nutrient entry, test mode, global skip, event logging
4. Document Segmentation and Core Extraction Process	Next phase	This report focuses on the first three working items

The system is organized around four ideas: a crawler that selects stronger candidates, a shared backend and data model, an annotation screen designed for work on top of the PDF, and a closed loop that turns user decisions into feedback.

Table 2 summarizes that structure through the main system parts.

Layer	Work completed	Technical result
Crawler and paper acquisition	Multi-source search, multi-stage filtering, EN/TR workflows, DergiPark support, PDF validation	Papers entering the system became more selective and more meaningful
Backend and data model	Shared database schema, search-evidence records, annotation tables, RLS policies	The UI, crawler, and feedback layer now operate on the same data structure
Annotator and user workflow	PDF viewing, highlight-assisted entry, dynamic food/nutrient form, test mode, global skip	Expert users can work on real documents faster and with better control
Learning feedback loop	Storing user decisions as	The system moved toward a

	events and reusing them in later retrieval	real closed loop that learns from user behavior
--	--	---

Figure 1 shows the end-to-end system state available at the midterm stage.

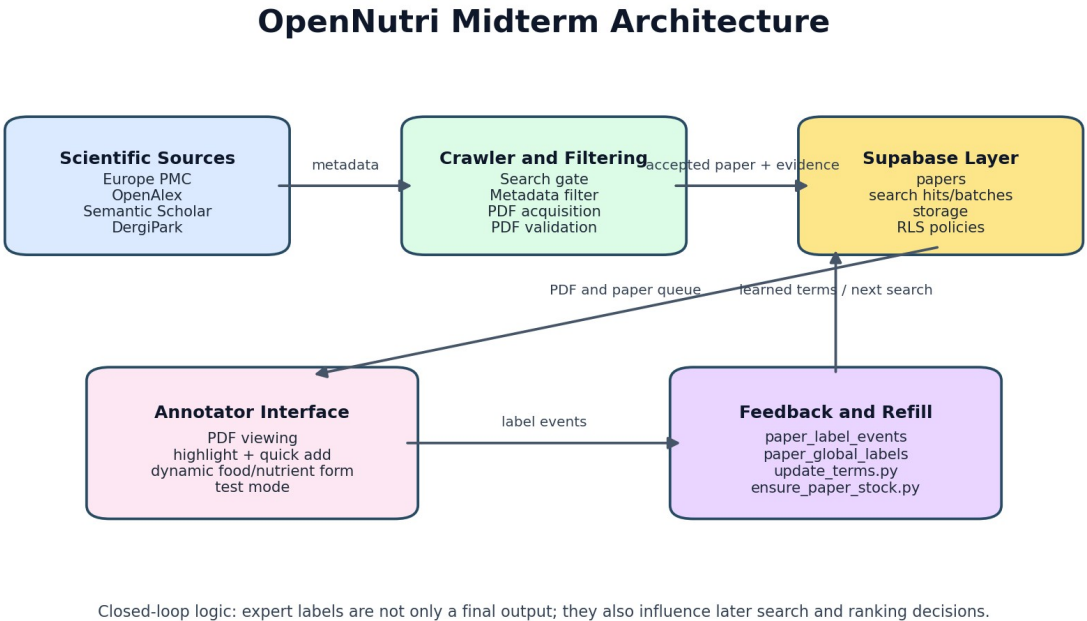


Figure 1. Closed-loop OpenNutri architecture implemented by the midterm stage.

At the midterm stage, the first three goals already exist as working system components. The current state is an early production-stage system with an operating core infrastructure.

PROJECT OBJECTIVE and IMPORTANCE

Project Objective

The objective of OpenNutri is to digitize food-composition data scattered across scientific literature through a platform centered on expert feedback. To achieve that goal, the project establishes three capabilities:

- automatically finding relevant papers and bringing them into the system,
- allowing experts to annotate those papers in a controlled way,
- using the resulting labels to improve later retrieval and ranking decisions.

At the midterm stage, the focus is to build the core version of these three capabilities. The current work establishes the data infrastructure, user workflow, and feedback loop.

Project Importance

The importance of the project appears at three levels.

The first level is the data-access problem. Food-composition studies are usually published as PDF documents across different journal infrastructures and rarely provide a standard machine-readable output. OpenNutri targets that gap.

The second level is efficient use of expert time. Expert annotation is expensive and limited. The crawler, filtering, and feedback layers are therefore designed to deliver more meaningful candidates to the annotator. At the midterm stage, search gate, metadata filter, PDF validation, and global skip logic already support this goal.

The third level is the visibility of Turkish literature. PubMed Central brings international open-access literature into the system, while DergiPark provides access to studies published in Turkey. The EN/TR workflow split establishes Turkish literature as a separate target pool.

From a technical point of view, the project combines a normalized data model, multi-source search, layered filtering, PDF validation, user-event logging, and soft feedback learning in one system. This structure creates a strong base for later document segmentation and extraction work.

LITERATURE REVIEW

The literature review examined both food-data standards and scientific-document processing tools. It covered academic papers, databases, open-access archives, ontologies, and the software ecosystems used in the implementation.

1. Food data and reference vocabularies

OpenNutri's data model is built around a shared vocabulary. FoodData Central [1] and the FAO/INFOODS matching approach [2] were reviewed, and canonical foods and canonical nutrients were modeled as separate tables. As a result, the food and nutrient choices in the UI and the terms used by the crawler rely on the same vocabulary.

Ontology-centered work such as FoodOn [3] is relevant for standardizing food names. The `entities` and `entity_aliases` structure was chosen for the same standardization need.

2. Scientific literature sources

PubMed Central [4] and Europe PMC [5] were evaluated as core external sources because they provide structured access to open-access biomedical and life-science literature. DergiPark [6] was included specifically to reach Turkish-language studies. During the midterm period, the DergiPark path was redesigned from a simple broad search approach into a renewable local journal/issue/article index. That change made Turkish-language crawling more controlled and auditable.

3. Human-in-the-loop learning approach

Human-in-the-loop methodology [7] keeps the expert user as an active part of extraction and verification. In OpenNutri, this approach is implemented through the annotator UI and `paper_label_events`. User actions such as `draft`, `done`, `skipped`, and `global_definitely_no_data` directly become crawler feedback.

4. PDF processing and UI infrastructure

Because article content is usually distributed as PDF, in-browser PDF processing became essential. PDF.js [8] and React [9] were evaluated together. The nutrient-highlighting feature required extra scanning logic because the PDF text layer is fragmented across spans.

5. Platform components used in the implementation

Supabase [10] was selected because it combines authentication, row-level access control, file storage, and client access in one platform. At the midterm stage, both the annotator and the data pipeline already share this backend.

The literature review led to several engineering decisions:

- separating paper discovery, pre-filtering, and PDF acquisition,
- designing the UI with a dynamic row model,
- storing labels as event history,
- using separate target pools for Turkish and English sources,
- using feedback as a soft scoring signal.

All of these decisions are already reflected in the current codebase.

MATERIALS AND METHODS

1. Overall system approach

The midterm version of OpenNutri consists of three major layers:

- the user-facing annotator interface,
- the shared backend and data model,
- the multi-source crawler and feedback layer.

The interaction of these layers was shown in Figure 1. Figure 2 expands that view by focusing on the internal data-model and feedback relationships.

Annotation and Feedback Data Relations

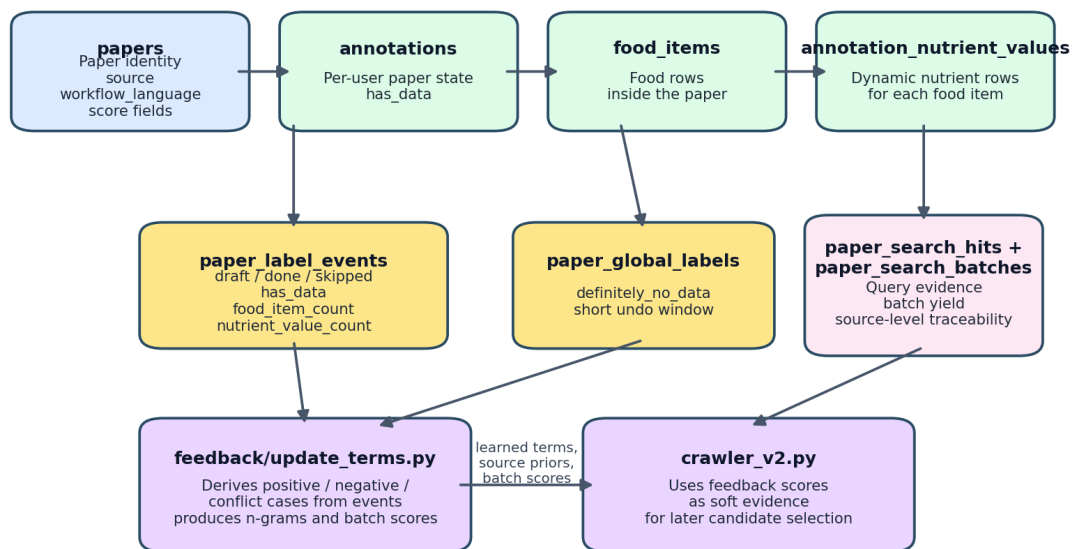


Figure 2. Data-model relationships linking annotation, event logging, and crawler feedback in the midterm system.

2. Materials used

The main technical materials used in the project are:

- **Frontend framework:** React 19 + Vite
- **Backend and storage:** Supabase Auth, PostgreSQL, Storage
- **PDF handling:** react-pdf and PDF.js
- **Data-pipeline language:** Python
- **Reference data source:** USDA FoodData Central [1]
- **Paper sources:** Europe PMC [5], PubMed Central [4], OpenAlex, Semantic Scholar, DergiPark [6]
- **Embedding layer:** a dual English + multilingual setup based on sentence-transformers

These components are used across two connected sides of the project: the web application that manages expert workflow and the data pipeline that feeds the paper queue.

3. Backend and data-model method

The main backend decision was to organize the system around one shared data model. That model can be understood through four parts: the shared food/nutrient vocabulary, the papers and search evidence entering the system, the expert annotation records, and the event logs that preserve user decisions as feedback.

This structure keeps the UI, crawler, and feedback logic on the same data model. The names used in the UI and the terms reused by the crawler rely on the same reference vocabulary. The reason a paper entered the system and the later annotation built on top of it are tied to the same paper record.

Figure 3 presents a simplified database summary derived from `apps/expert-annotator/migration.sql`, which is the code-level source of truth for the active Supabase schema.

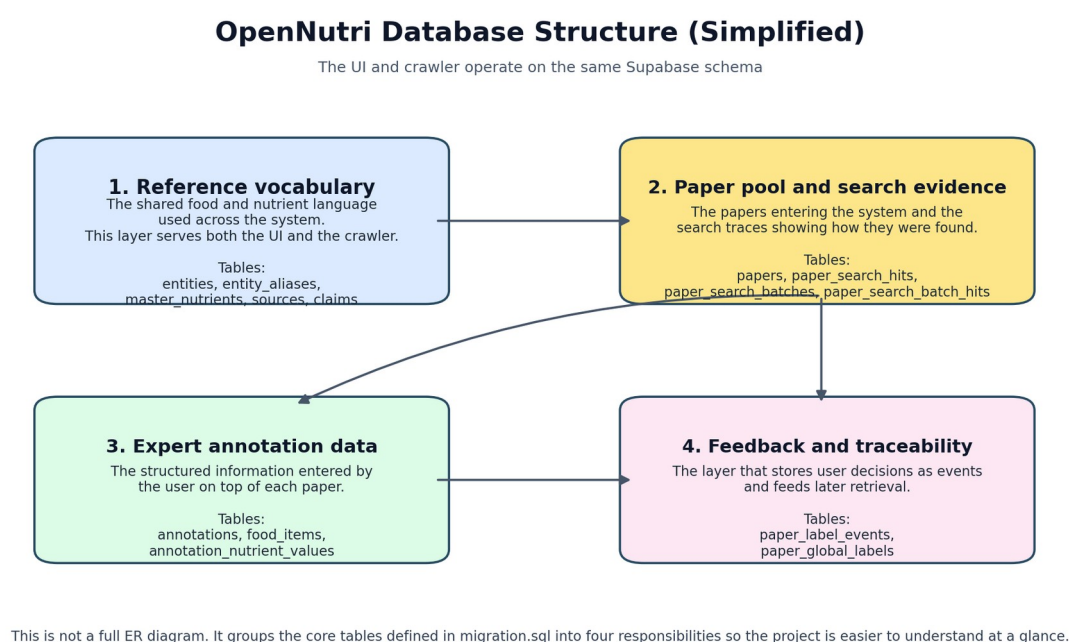


Figure 3. Simplified view of the main database structure used in the midterm system, grouped into four responsibilities so the project is easier to understand. The summary is derived from the current `migration.sql` schema definition.

Row-level security (RLS) is used to protect the backend. Users can manage their own annotation data, while the service role remains able to perform crawler uploads, ETL, and maintenance operations. This is the core security mechanism required by the system's multi-user design.

4. Annotator interface method

The annotator interface is designed as a working screen where the expert can read a paper and enter structured data at the same time. The user opens a paper from the system queue, sees any previously saved work, and continues from the last meaningful state.

The interface follows two principles. The user can add as many food items and nutrient values as the paper requires. The PDF and the form are presented in the same working screen so the user can read the document and move quickly into structured entry.

Three interface behaviors are especially important:

- a test mode that allows safe use of the full UI without writing to the real database,
- a global "definitely no data" action with a short undo window,
- counting valid food items to prevent empty placeholder cards from inflating food_item_count.

Figure 4 shows the numeric summary produced by the crawler's staged flow.

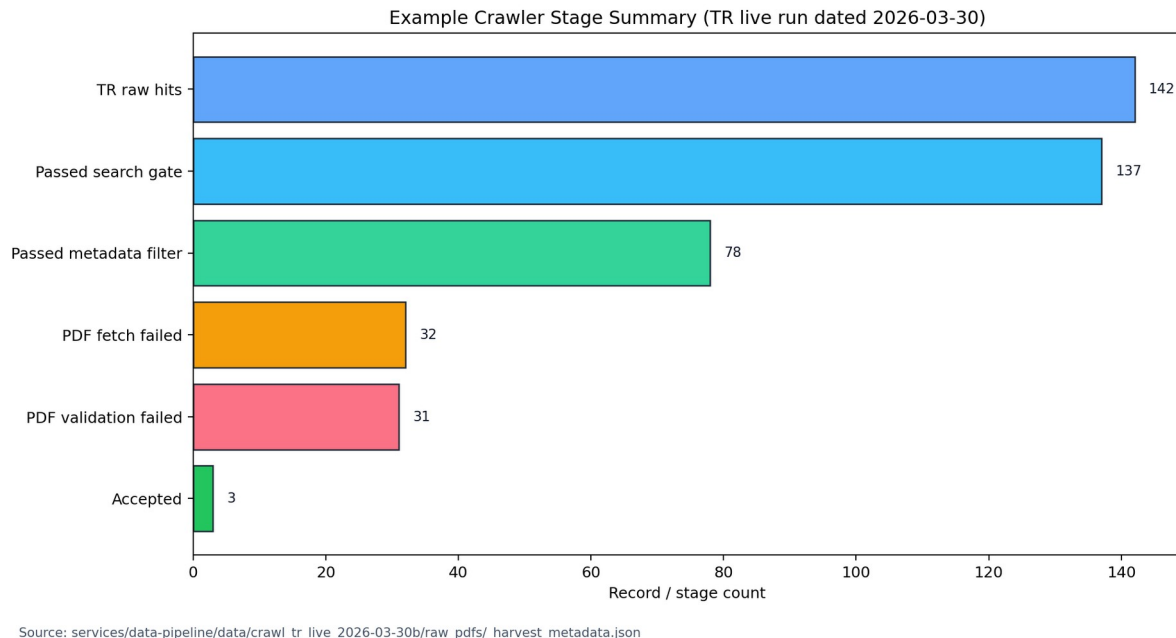


Figure 4. Stage counts derived from the manifest summary of a representative Turkish live run.

Figure 5 is the placeholder for the final annotator UI screenshot.

Placeholder for Figure 5

This visual should be replaced with a real annotator screenshot

1. In apps/expert-annotator run npm install && npm run dev.
2. Sign in and open the Annotate page.
3. Make sure the same frame includes:
PDF viewer, a highlighted nutrient, the right-side food item form,
and the progress / status area near the top.
4. Save the screenshot under the same filename as
docs/defense/assets/figure_5_annotator_placeholder_en.png
and rerun the export script.

Figure 5. Before final submission, this visual should be replaced with a real screenshot of the current annotator interface. The simplest approach is to replace docs/defense/assets/figure_5_annotator_placeholder_en.png with the screenshot

under the same filename and rerun the export script. The image should include the PDF viewer, an example highlighted nutrient, the food-item form, and the progress/status area in the same frame.

5. Crawler, filtering, and acquisition method

The crawler is built as a staged selection pipeline. The basic approach has three steps:

- **Search:** finding metadata-level candidates from sources such as Europe PMC, OpenAlex, Semantic Scholar, and DergiPark
- **Filter:** applying search-gate and metadata-filter logic to titles and abstracts
- **Acquisition:** downloading PDFs and validating full text only for sufficiently strong candidates

This separation reduces unnecessary PDF downloads and moves fewer, stronger candidates into full-text acquisition.

The filtering stage combines three signal groups: domain-specific keyword and unit clues, semantic suitability based on embeddings, and feedback signals learned from previous user behavior.

Two crawler capabilities are especially important at this stage: separate target pools for English and Turkish literature, and feedback reuse at both term and query-batch level.

For Turkish-language acquisition, the DergiPark integration was also redesigned. Instead of a broad and weakly controlled search approach, the crawler now uses renewable journal- and issue-level local index files. This improves both source quality and traceability for Turkish literature.

6. Feedback and paper-stock refill method

The feedback/update_terms.py script reads user-created event records and turns them into learning signals. Cases with meaningful saved data are treated as positive. Papers that clearly contain no relevant data or are repeatedly skipped are treated as negative. Mixed cases are kept out of training.

The resulting feedback is then consumed by the crawler as a soft scoring signal.

When the end-user paper pool becomes too small, ensure_paper_stock.py takes over. This script checks the current EN/TR paper counts, refreshes feedback when needed, refreshes the DergiPark index, runs the crawler, and uploads the results to Supabase. That creates an operational bridge between the annotation interface and the data-acquisition layer.

7. Current limitations

This report focuses on the first three working semester goals and the data/feedback infrastructure supporting them. Document segmentation and the LLM-based extraction process belong to the next phase.

The annotator screenshot is still left as a placeholder in this report. It should be replaced with the latest UI state before final submission.

REFERENCES

1. U.S. Department of Agriculture. FoodData Central. <https://fdc.nal.usda.gov/>
2. FAO/INFOODS. Guidelines for Food Matching. Rome: Food and Agriculture Organization of the United Nations, 2012.
3. Dooley DM, Griffiths EJ, Gosal GS, et al. FoodOn: a harmonized food ontology to increase global food traceability, quality control and data integration. **npj Science of Food**. 2018;2(1):23.
4. National Center for Biotechnology Information. PubMed Central (PMC). <https://pmc.ncbi.nlm.nih.gov/>
5. Europe PMC. <https://europepmc.org/>
6. DergiPark Akademik. <https://dergipark.org.tr/>
7. Monarch RM. **Human-in-the-Loop Machine Learning**. Manning Publications; 2021.
8. Mozilla. PDF.js. <https://mozilla.github.io/pdf.js/>
9. React Documentation. <https://react.dev/>
10. Supabase Documentation. <https://supabase.com/docs>